

Cloud-to-Edge Integration

INSISN – Integração de Sistemas e Serviços na Nuvem

António Barros

Departamento de Engenharia Informática

Virtual Networking in Virtualised Environments

Introduction to Software-Defined Networking (SDN)

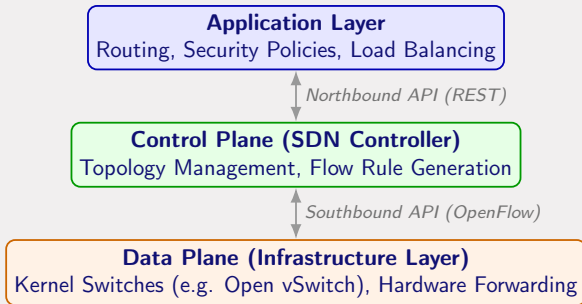
- **Transition from Hardware to Software Switching:**
 - ▶ Traditional setups rely on physical Layer 2/3 hardware switches.
 - ▶ Virtualisation replaces hardware frames with software-based packet processing within the Host Kernel.
- **Decoupling Control and Data Planes:**
 - ▶ **Control Plane:** Programmatic rule definition, topology management, and forwarding tables.
 - ▶ **Data Plane:** Fast packet inspection, filtering, and switching executed at the kernel level.
- **Key Advantages in Local Environments:**
 - ▶ Dynamic provision of complex topologies without physical rewiring.
 - ▶ Automated isolation, micro-segmentation, and fine-grained access control.

SDN Architecture: Practical Execution Example

- Control Plane (Centralised Intelligence)
 - ▶ **SDN Controller:** Software application running on a dedicated host/VM.
 - ▶ **Function:** Calculates paths, defines security policies, and injects flow rules via OpenFlow/Southbound APIs.
- Data Plane (High-Speed Packet Processing)
 - ▶ **Virtual Switches (e.g. Open vSwitch):** Running inside the host kernel across multiple physical servers.
 - ▶ **Function:** Fast, rule-based packet inspection and forwarding strictly following controller instructions.

Adding a new micro-segmentation rule update requires configuring **one single controller**, which instantly propagates rules across all hypervisor kernel switches.

SDN Layered Architecture



Decoupling is achieved through standard interfaces: the **Control Plane** translates high-level application logic into low-level forwarding rules applied at the **Data Plane**.

The Linux Bridge Architecture (vbr0)

- **Virtual Layer 2 Transparent Switch:**

- ▶ Operates as a software-based IEEE 802.1D Ethernet bridge inside the Linux kernel.
- ▶ Inspects incoming frame headers to build and maintain a dynamic MAC address table (fdb).

- **Frame Forwarding Mechanics:**

- ▶ **Unicast:** Frames targeting a known MAC address are forwarded strictly to the assigned virtual port.
- ▶ **Broadcast / Unknown Unicast:** Flooded across all active attached ports.

- **Integration in Virtualisation Hosts:**

- ▶ Functions as the main virtual switch (e.g., vbr0 in Proxmox or default br0 constructs).
- ▶ Binds physical network interface cards (NICs) with virtual guest interfaces.

Virtual Network Interfaces: TAP vs. TUN

- **Kernel-to-Userspace Abstraction:**
 - ▶ Software-based interfaces allowing userspace programs (e.g., QEMU/KVM) to inject and capture packets directly.
- **TAP Devices (Layer 2 - Ethernet Frames):**
 - ▶ Operates at the Data Link layer; processes raw Ethernet frames including MAC headers.
 - ▶ **Primary Use Case:** Virtual Machines (KVM/QEMU) requiring complete L2 transparency, custom MAC addresses, and non-IP protocols.
- **TUN Devices (Layer 3 - IP Packets):**
 - ▶ Operates at the Network layer; strips Ethernet frames and processes pure IP packets.
 - ▶ **Primary Use Case:** Point-to-point tunnels, VPN daemons (e.g., OpenVPN, WireGuard), and routed topologies.

Virtual Ethernet Pairs (veth)

- **The Virtual Cable Metaphor:**

- ▶ Always created as bidirectional connected pairs (vethA ↔ vethB).
- ▶ Packets transmitted into one end instantly emerge at the opposing end.

- **Cross-Namespace Communication:**

- ▶ Essential mechanism for container isolations (LXC, Docker, Podman).
- ▶ One end stays attached to the host bridge inside the default network namespace; the peer is moved into the container's isolated namespace.

Packet Path: Container to External Network

Container App → Container eth0 $\xrightarrow{\text{veth pair}}$ Host vethX → Linux Bridge (vbr0) → Physical NIC

Physical Interface Modes: Bridged, Routed and NAT

Bridged Mode

- Guests share the physical NIC's broadcast domain.
- Guests request IPs directly from the external physical DHCP.
- Full L2 transparency across the LAN.

Routed Mode

- Host acts as a Layer 3 router.
- Guests are assigned IPs from an isolated internal subnet.
- Requires static routing rules on upstream routers.

NAT Mode

- Host acts as gateway with IP masquerading.
- Guest outbound traffic translates to Host's physical IP.
- Inbound access requires explicit Port Forwarding.

Physical Interface Modes: Suitable Use Cases

- **Bridged Mode:**
 - ▶ **Production Servers:** Ideal for Web or Database servers requiring direct LAN visibility.
 - ▶ **Seamless Integration:** Best when VMs must obtain IP addresses directly from the enterprise DHCP/router.
- **Routed Mode:**
 - ▶ **Data Centre Hosting:** Suitable when providers allocate dedicated public IP blocks to a host.
- **NAT Mode:**
 - ▶ **Development & Testing:** Perfect for rapid prototyping and local lab environments.

VLANs in Virtual Bridges: Layer 2 Isolation

- **The Need for Traffic Segmentation:**

- ▶ Multi-tenant virtual hosts require strict isolation between co-located Virtual Machines.
- ▶ Unfiltered Layer 2 environments expose traffic to eavesdropping and broadcast storms.

- **IEEE 802.1Q Frame Tagging:**

- ▶ Inserts a 4-byte VLAN header containing a 12-bit VLAN ID (VID) into the Ethernet frame.
- ▶ Enables network devices to logically segregate traffic without physical infrastructure changes.
- ▶ Guarantees that VMs on different VIDs cannot communicate directly at Layer 2.

Virtual Bridge Architecture: Traditional vs. VLAN-Aware

Traditional Approach (Multi-Bridge Model)

- Requires creating a dedicated virtual bridge per VLAN segment (e.g., `vmbr0.10`, `vmbr0.20`).
- High administrative overhead and complex configuration files on the host.

Modern Approach (VLAN-Aware Bridge)

- Operates a single unified bridge (`vmbr0`) capable of internal 802.1Q frame filtering.
- VLAN tags are assigned directly to the virtual network interface cards (vNICs) of the guest VMs.
- Significantly simplifies network design and host configuration management.

VLAN Trunking to Physical Infrastructure

- **Bridging Virtual and Physical Segments:**

- ▶ The physical host interface (NIC) acts as an 802.1Q trunk port connected to a physical switch.
- ▶ Preserves VLAN tags when traffic exits the virtual host onto the physical wire.

Operational Benefit

Enables seamless extension of enterprise network security policies directly down to individual Virtual Machines.

Advanced Addressing: Motivation & Efficiency

- **Limitations of Traditional Virtual Bridges:**

- ▶ Standard bridges (vbr0, docker0) emulate full Layer 2 switches in software.
- ▶ Introduce overhead: packet inspection, MAC table maintenance (fdb), and spanning tree calculations.

- **Direct Binding (Bypassing the Switch):**

- ▶ Drivers like MACVLAN and IPVLAN attach virtual interfaces directly to the physical network card (NIC).
- ▶ Elimination of virtual bridges reduces CPU overhead, memory footprint, and packet traversal latency.

- **Primary Target Use Case:**

- ▶ High-density container environments (Docker, LXC, Podman) requiring near-native network throughput.

MACVLAN: Unique MACs per Interface

- **Core Mechanism:**

- ▶ Binds multiple virtual sub-interfaces to a single physical NIC.
- ▶ Assigns a unique, dedicated **MAC address** and **IP address** to each guest or container.
- ▶ To external physical switches, guests appear as distinct physical devices on the wire.

- **Operational Modes:**

- ▶ **Private / VEPA:** Restricts direct inter-guest communication at the host level.
- ▶ **Bridge Mode:** Allows containers on the same host to communicate directly at Layer 2.

- **Key Limitation:**

- ▶ Requires **promiscuous mode** on the host NIC; frequently blocked by cloud providers (AWS/Azure) and enterprise switch port-security policies.

IPVLAN: Shared MAC with Independent IPs

- **Core Mechanism:**

- ▶ All endpoints **share the exact same MAC address** as the host's physical NIC.
- ▶ Each guest maintains its own independent IP address.
- ▶ Avoids promiscuous mode, making it fully compatible with restricted physical switches and cloud environments.

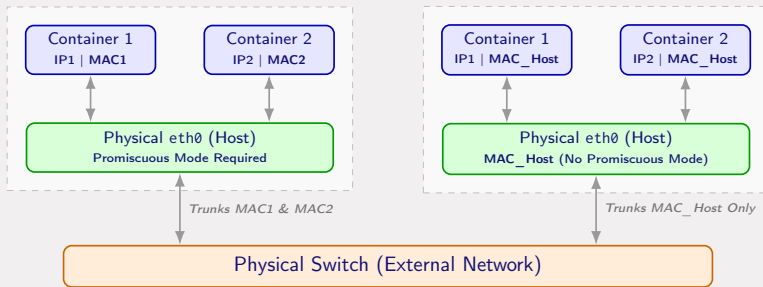
- **Operating Modes:**

- ▶ **L2 Mode:** Processes ARP and IP traffic; acts like a traditional Layer 2 bridge without MAC proliferation.
- ▶ **L3 Mode:** Host acts as a pure Layer 3 routing engine; drops broadcast/multicast for maximum scalability.

- **Primary Use Case:**

- ▶ Massive-scale container deployments where MAC address table limits on physical switches are a concern.

Visual Comparison: MACVLAN vs. IPVLAN



Network Performance and Hardware Offloading

- **Virtual Switching Overhead:**

- ▶ Processing packet frames in software incurs high CPU interrupts, context switching, and memory copies.

- **Hardware Offloading Mechanisms:**

- ▶ **Checksum Offloading (Tx/Rx):** Delegates TCP/UDP checksum calculations to physical NIC hardware.
- ▶ **TSO (TCP Segmentation Offloading) / LRO (Large Receive Offloading):** Allows the kernel to process large buffers (up to 64KB) before hardware breaks them into standard 1500-byte MTUs.

- **SR-IOV (Single Root I/O Virtualisation):**

- ▶ Bypasses the Hypervisor/Kernel network stack completely.
- ▶ Exposes physical PCIe Virtual Functions (VF) directly inside VMs for near-native line rate performance.

Introduction to Containers

A High-Level Conceptual Overview

The "Works on My Machine" Dilemma

- **Software Complexity:** Modern applications rely on diverse runtime environments, libraries, and configurations.
- **Environment Drift:** Differences between development, testing, and production lead to unexpected bugs.
- **Deployment Overhead:** Traditional provisioning is slow and error-prone.
- **The Core Need:** A consistent, predictable unit of software deployment.

Evolution of Infrastructure

1. Physical Servers (Bare Metal)

Single OS per machine, poor resource utilisation, long setup times.

2. Virtual Machines (VMs)

Hardware-level abstraction. Multiple OSs on one physical machine, but resource-heavy.

3. Containers

OS-level abstraction. Lightweight, near-instant start-up, highly portable.

Virtual Machines vs Containers

Feature	Virtual Machines	Containers
Abstraction	Hardware level	OS level (Kernel sharing)
Start-up Time	Minutes	Seconds / Milliseconds
Resource Footprint	Gigabytes (Heavy)	Megabytes (Lightweight)
Isolation	Full hardware isolation	Process-level isolation

What is a Container?

- **Definition:** A standardized, isolated package containing an application and all its dependencies.
- **Key Characteristics:**
 - ▶ **Lightweight:** Shares the host system kernel.
 - ▶ **Portable:** Runs identically on any cloud or local environment.
 - ▶ **Isolated:** Processes and file systems are kept separate from other containers.

Blueprints vs Reality: Images vs Containers

Container Image

- The **blueprint** or template.
- Read-only file system layer.
- Contains code, binaries, runtime, and settings.
- Immutable by design.

Container Instance

- The **running process**.
- Created from an image.
- Adds a thin read/write layer on top.
- Ephemeral (temporary state).

The Container Lifecycle

1. **Build:** Define the application components into an image specification.
2. **Ship:** Store and share the image in a central registry.
3. **Run:** Instantiate and execute the container on any compatible host.

Under the Hood: How Isolation Works

Containers leverage fundamental OS-level constructs (primarily in Linux):

- **Namespaces:** Provide *isolation* (what a process can **see**).
 - ▶ Separate process IDs, network interfaces, and file system mounts.
- **Control Groups (cgroups):** Provide *resource control* (what a process can **use**).
 - ▶ Limit and monitor CPU, memory, and I/O usage per container.

The Container Ecosystem

- **Engine / Runtime:** Manages image building and container execution (e.g., Docker, containerd, Podman).
- **Registries:** Central repositories for storing and distributing images (e.g., Docker Hub, GitHub Packages).
- **Standards:** Open Container Initiative (OCI) ensures interoperability across different tools.

Data Management: Ephemeral vs Persistent

- **Default Behaviour:** Container storage is ephemeral. Data vanishes when the container is destroyed.
- **The Solution (Volumes):** Decouple data storage from the container lifecycle.
- **Stateless Applications:** Ideal for containers (web servers, microservices).
- **Stateful Applications:** Require explicit persistent storage strategies (databases).

Scaling Up: The Need for Orchestration

Single Container

- Easy to manage manually.
- Great for development.

Production Scale (100s of Containers)

- How to handle host failures?
- How to scale up during traffic peaks?
- How to manage network routing?

Container Orchestration

Tools like **Kubernetes** and **Docker Swarm** automate deployment, scaling, healing, and management of multi-container systems.

Benefits and Considerations

Key Benefits

- Consistency across environments.
- Fast startup and low overhead.
- Efficient hardware usage.
- Foundation for Microservices.

Key Considerations

- Shared kernel security boundaries.
- Operational complexity at scale.
- Storage/State management planning.

Containers in the Cloud

How Containers Drive Modern Cloud Architectures

The Perfect Match: Containers & Cloud

- **Cloud Elasticity + Container Agility:** Cloud provides scalable infrastructure; containers provide lightweight application packaging.
- **Write Once, Run Anywhere:** Seamless mobility between local environments, public clouds, and multi-cloud setups.
- **Resource Density:** Maximises hardware utilization compared to traditional Virtual Machines (VMs) in the cloud.
- **Microservices Foundation:** Enables cloud-native architectures built from decoupled, independent services.

Deployment Models in the Cloud

Model	Mechanism	Examples
IaaS (Infrastructure)	Self-managed on Cloud VMs	AWS EC2, Azure VMs, Compute Engine
Managed Orchestration	Managed Kubernetes Control Planes	AWS EKS, Azure AKS, Google GKE
Serverless Containers	Event-driven, zero server management	AWS Fargate, Google Cloud Run

Cloud-Native Orchestration (Kubernetes)

- **The Scale Challenge:** Managing thousands of containers manually in the cloud is impossible.
- **Key Responsibilities of Cloud Orchestrators:**
 - ▶ **Auto-scaling:** Adjusting container instances based on real-time traffic demand.
 - ▶ **Self-healing:** Automatically restarting failed containers or re-routing traffic.
 - ▶ **Load Balancing:** Distributing incoming requests across healthy container replicas.
- **Managed Kubernetes Services:** Cloud providers remove the operational burden of managing the control plane.

Traditional Container Hosting: Managing the Cluster

- **Infrastructure Management:** Requires explicit provisioning and sizing of virtual machines (nodes) to host the containers.
- **Fixed Cost Model:** You pay for the pre-allocated CPU and RAM capacity 24/7, regardless of actual application traffic.
- **Operational Complexity:** Scaling requires configuring complex auto-scaling rules for both container replicas and underlying VM nodes.
- **Best Suited For:** Large-scale, predictable enterprise workloads with consistent resource demands.

Serverless Containers: The Next Evolution

- **Zero Infrastructure Overhead:** The cloud provider completely abstracts the servers; you only supply the container image.
- **True Pay-per-Use:** Billing is calculated strictly per millisecond of compute time active during request execution.
- **Scale to Zero:** Automatically spins down container instances when idle, resulting in zero cost during zero-traffic periods.
- **Modern Cloud Implementations:** Exemplified by AWS Fargate, Google Cloud Run, and Azure Container Apps.

Containers in Cloud CI/CD Pipelines

1. **Code Commit:** Developer pushes code to source control.
2. **Automated Build:** Cloud pipeline builds a immutable container image.
3. **Registry Storage:** Image is scanned for security and pushed to a Cloud Container Registry (e.g., ECR, ACR, GAR).
4. **Zero-Downtime Deployment:** Orchestrator rolls out the new container version seamlessly using rolling updates or blue-green strategies.

Cloud Storage & Networking Integration

- **Cloud Volumes (Persistent Data):**

- ▶ Attaching persistent cloud block storage (e.g., AWS EBS, Azure Disk) to ephemeral containers.

- **Cloud Networking and Ingress:**

- ▶ Integrating container networks directly with Cloud Virtual Private Clouds (VPCs).
- ▶ Exposing services using Cloud Native Load Balancers and Ingress Controllers.

Security and Multi-Cloud Strategy

Cloud Container Security

Image vulnerability scanning, fine-grained Identity & Access Management (IAM), and network isolation policies.

Avoiding Vendor Lock-in

Containers abstract the underlying cloud infrastructure. An application packaged in a standard OCI container runs identically on AWS, Azure, Google Cloud, or On-Premises.